Bölüm 16: Ödev 1 – Kavrayış özeti

SE 422 · Introduction to Data Science · Konu özeti

Kaynak slayt destesi: Homework 1
7 Kasım 2026

İçindekiler

Bölüm 16 · Ödev 1 – Kavrayış özeti 3

Bölümün amacı 3

Konu anlatımı 3

Özyineleme ve ikili gösterim (Task 1) 3

Vector sınıfı ve dunder metotlar (Task 2) 4

CPython ve küçük tamsayı önbelleği (Task 3) 5

Kesme ve yuvarlama (Task 4) 7

Broadcasting, toplamalar ve z-skoru (Task 5) 8

Sık yapılan hatalar 9

Kısa tekrar soruları 10

Cevaplar 11

Bölüm 16 · Ödev 1 – Kavrayış özeti

> ⚠ *Yapay zekâ ile üretildi; slayt ve ödev metniyle karşılaştır.*

> *Bu özet ödevin **çözümü değildir**; ödevin "Analysis" kısımlarında beklenen kavrayışı sınav için toparlar. Kod örnekleri kavramı göstermeye yetecek kadar kısadır. Sürüme veya ortama bağlı her çıktı bu ortamda (CPython 3.9.6, NumPy 2.0.2) çalıştırılarak üretildi; kendi sürümünde farklı çıkabilir.*

Bölümün amacı

Hoca Homework 1'in ara sınava gömüleceğini söyledi: sınav, ödevi yaparken edinmen gereken **kavrayışı** ölçer. Bu özetten sonra şunları açıklayabilmelisin:

- Özyinelemenin bellek modelini (her çağrı bir frame), özyineleme sınırını ve `RecursionError`'ı; döngü ve generator alternatiflerini
- `__add__`, `__radd__`, `__str__`, `__repr__` ve `NotImplemented` mekanizmasını; `sum()`'ın neden `__radd__` istediğini
- CPython, `PyLongObject`, -5…256 küçük tamsayı önbelleği; `is` ile `==` farkı; REPL ile betik farkı
- `int()` ile kesme, `round()` ile banker's rounding, kayan nokta temsil sorunları ve NumPy karşılıkları
- Broadcasting kurallarını, `axis` ile toplamaları, `keepdims`'i ve sütun bazında z-skorunu

Konu anlatımı

Özyineleme ve ikili gösterim (Task 1)

`bin(n)` başında `0b` olan bir string, `format(n, 'b')` öneksiz string döndürür. `int(s, 2)` ikili stringi sayıya geri çevirir.

PYTHON
```
print(bin(5), format(5, 'b'), int('101', 2))
```

ÇIKTI
```
0b101 101 5
```

Özyinelemeli her çağrı, çağrı yığınına (call stack) yeni bir **frame** ekler. Derinliği n olan bir özyineleme O(n) yığın belleği kullanır, döngü ise tek frame'le O(1) ek bellek kullanır. Python kuyruk çağrısı optimizasyonu yapmaz. Derinlik `sys.getrecursionlimit()` değerini aşarsa `RecursionError` fırlatılır:

PYTHON
```python
import sys

def derinlik(n):
    return 0 if n == 0 else 1 + derinlik(n - 1)

print(sys.getrecursionlimit())
try:
    derinlik(5000)
except RecursionError as e:
    print("RecursionError:", e)
```

ÇIKTI
```
1000
RecursionError: maximum recursion depth exceeded in comparison
```

Alternatifler, aynı işi yapmanın önerilen yollarıdır: düz bir `for` döngüsü ya da değerleri **tembel** (lazy) üreten bir generator ifadesi. Generator değerleri istendikçe üretir ve bellekte tüm listeyi tutmaz:

PYTHON
```python
gen = (format(i, 'b') for i in range(4))
print(type(gen).__name__, list(gen))
```

ÇIKTI
```
generator ['0', '1', '10', '11']
```

Okunabilirlik açısından özyineleme, problem doğası gereği özyinelemeli olduğunda (ağaçlar, böl-yönet) daha açıktır. Hız ve bellek açısından ise Python'da döngü genellikle daha iyidir: bu ortamda özyinelemeli sürüm comprehension'dan yaklaşık 2 kat yavaş ölçüldü.

Vector sınıfı ve dunder metotlar (Task 2)

`a + b` ifadesi önce `a.__add__(b)` çağrısını dener. Bu çağrı `NotImplemented` döndürürse Python yansıtılmış metot `b.__radd__(a)`'yı dener. İkisi de çalışmazsa `TypeError` fırlatılır. `NotImplemented` bir hata değil, bir **sinyaldir** (`NotImplementedError` ile karıştırma).

`sum()` toplamaya 0'dan başlar. İlk adım `0 + v1` olduğu için `int.__add__` Vector'ü tanımaz ve `v1.__radd__(0)` gerekir:

PYTHON
```python
class V:
    def __init__(self, x, y):
        self.x, self.y = x, y
```

PYTHON (DEVAM)

```python
    def __add__(self, other):
        if isinstance(other, V):
            return V(self.x + other.x, self.y + other.y)
        return NotImplemented

try:
    sum([V(1, 2), V(3, 4)])
except TypeError as e:
    print("TypeError:", e)
```

ÇIKTI

```
TypeError: unsupported operand type(s) for +: 'int' and 'V'
```

`__radd__` 0'ı kabul edince `sum` çalışır. `__str__` kullanıcıya yönelik gösterimdir (`print`, `str`). `__repr__` ise geliştiriciye yöneliktir (REPL, listelerin içi) ve mümkünse nesneyi yeniden üretebilecek biçimde yazılır. `__str__` yoksa `print` `__repr__`'a düşer:

PYTHON

```python
class V:
    def __init__(self, x, y):
        self.x, self.y = x, y
    def __radd__(self, other):
        return self if other == 0 else NotImplemented
    def __repr__(self):
        return f"V({self.x}, {self.y})"

v = V(1, 2)
print(v, [v], 0 + v)
```

ÇIKTI

```
V(1, 2) [V(1, 2)] V(1, 2)
```

Ödevde tartışılması beklenen **uç durumlar** şunlardır: skalerle toplama (`v + 5` desteklenecek mi, yoksa `NotImplemented` mı dönecek?), farklı boyutlu vektörler (3B ile 2B toplanırsa ne olmalı?) ve `other` nesnesinin Vector olmaması.

CPython ve küçük tamsayı önbelleği (Task 3)

CPython, Python'un C ile yazılmış referans uygulamasıdır (python.org'dan indirilen yorumlayıcı). **PyLongObject**, CPython'da her Python `int` değerini temsil eden C yapısıdır. CPython başlangıçta -5…256 arasındaki tamsayı nesnelerini önceden oluşturur, bu aralıktaki her değer aynı nesneye bağlanır.

`==` **değer** eşitliğini, `is` ise **nesne kimliğini** test eder. Değerler çalışma zamanında üretildiğinde önbelleğin sınırı görünür hale gelir:

PYTHON

```python
import sys
print(sys.version.split()[0])
for v in ["-6", "-5", "256", "257"]:
    a, b = int(v), int(v)
    print(v, a == b, a is b)
```

ÇIKTI

```
3.9.6
-6 True False
-5 True True
256 True True
257 True False
```

Sabit katlama ve kod blokları: aynı kod bloğunda (betik, fonksiyon) geçen aynı sabitler derleyicide tek nesneye birleşebilir. Bu yüzden betikte `a = 257; b = 257` için `a is b` True çıkabilir. REPL'de ise her satır ayrı derlenir ve sonuç False olur. Ödevin sayıları `int("257")` gibi **çalışma zamanında** üretmeyi istemesinin sebebi budur. Aşağıda iki durum `compile()` ile taklit ediliyor:

PYTHON

```python
ns = {}
exec(compile("a = 257\nb = 257", "<betik>", "exec"), ns)
print("aynı blok:", ns["a"] is ns["b"])
exec(compile("a = 257", "<1>", "single"), ns)
exec(compile("b = 257", "<2>", "single"), ns)
print("ayrı satırlar:", ns["a"] is ns["b"])
```

ÇIKTI

```
aynı blok: True
ayrı satırlar: False
```

Önbellek aralığı ve derleyici davranışı sürüme ve uygulamaya bağlıdır (PyPy gibi uygulamalar farklı davranır). Bu yüzden sayılar her zaman `==` ile karşılaştırılmalı, `is` yalnızca `None` gibi tekil nesneler için kullanılmalıdır.

NumPy uygun değildir: NumPy dizileri Python `int` nesneleri değil, ham makine değerleri tutar. Her erişimde yeni bir `np.int64` skaleri oluşturulduğu için `is` CPython önbelleği hakkında bir şey söylemez:

PYTHON

```python
import numpy as np
a = np.array([5, 5])
print(type(a[0]).__name__, a[0] is a[1])
```

ÇIKTI
```
int64 False
```

Kesme ve yuvarlama (Task 4)

`int()` **sıfıra doğru keser**, yani ondalık kısmı atar. `round()` ise en yakın tamsayıya yuvarlar ve tam .5 olduğunda **en yakın çift** sayıyı seçer (round-half-to-even, banker's rounding):

PYTHON
```
print(int(-2.7), int(2.7), round(-2.7))
print(round(0.5), round(1.5), round(2.5), round(-0.5))
```

ÇIKTI
```
-2 2 -3
0 2 2 0
```

Bias (sapma): "yarıyı hep yukarı yuvarla" kuralı çok sayıda .5'li değerde toplamı sistematik olarak büyütür. Çifte yuvarlamada yarılar yarı yarıya aşağı ve yukarı gittiği için ortalama sapma sıfıra yaklaşır. Kesme ise pozitif sayıları hep küçültür ve sapması büyüktür. Para sistemlerinde bu yüzden yuvarlama stratejisi bilinçli seçilir.

Kayan noktalı sayılar ikili sistemde çoğu zaman tam temsil edilemez. `2.675` aslında 2.67499999... olarak saklanır, bu yüzden `round(2.675, 2)` 2.68 değil 2.67 verir. Bu bir hata değildir. Tam ondalık hesap gerekiyorsa `decimal` modülü kullanılır.

PYTHON
```
print(round(2.675, 2), f"{2.675:.20f}")
```

ÇIKTI
```
2.67 2.67499999999999982236
```

Tekrarlanabilir rastgele veri için `random.seed(...)` (NumPy'da `np.random.seed` ya da `np.random.default_rng(seed)`) kullanılır. Aynı seed aynı diziyi üretir.

NumPy karşılıkları şunlardır: `np.trunc` (kesme), `np.rint` ve `np.round` (çifte yuvarlama). Sonuçlar float döner. **Vektörleştirme** döngüyü C'de, Python nesnesi oluşturmadan yapar:

PYTHON
```
import numpy as np
x = np.array([-2.7, -0.5, 0.5, 1.5, 2.5])
print(np.trunc(x), np.rint(x))
```

ÇIKTI

```
[-2. -0.  0.  1.  2.] [-3. -0.  0.  2.  2.]
```

Bu ortamda 10^6 değer üzerinde ölçüldü: `round()`, `int()`'ten yaklaşık 1,1 kat yavaş. Liste üzerinde `int` ile `np.trunc` karşılaştırmasında NumPy yaklaşık 130 kat, liste üzerinde `round` ile `np.rint` karşılaştırmasında yaklaşık 360 kat hızlı. Bu oranlar makineye göre değişir, sıralama ise genellikle aynı kalır.

Broadcasting, toplamalar ve z-skoru (Task 5)

Broadcasting kuralları: iki dizinin şekilleri **sağdan** karşılaştırılır. Eksik boyutlar soldan 1 kabul edilir. Her boyutta değerler ya eşit olmalı ya da biri 1 olmalıdır, aksi halde `ValueError` fırlatılır.

- (3, 4) + (4,) → (4,) önce (1, 4) olur, satırlara yayılır → (3, 4)
- (3, 4) + (3, 1) → sütun vektörü sütunlara yayılır → (3, 4)
- (3, 4) + (3,) → (3,) önce (1, 3) olur, son boyutlar 4 ile 3 uyuşmaz → `ValueError`

PYTHON

```python
import numpy as np
A = np.arange(1, 13).reshape(3, 4)
print((A + np.array([10, 20, 30, 40])).shape, (A + np.array([1, 2, 3]).reshape(
↪ 3, 1)).shape)
print(np.array([1, 2, 3])[:, np.newaxis].shape)
try:
    A + np.array([1, 2, 3])
except ValueError as e:
    print("ValueError:", e)
```

ÇIKTI

```
(3, 4) (3, 4)
(3, 1)
ValueError: operands could not be broadcast together with shapes (3,4) (3,)
```

axis: `axis=0` satırlar boyunca indirger, yani **her sütun** için bir sonuç verir. `axis=1` **her satır** için bir sonuç verir. (5, 4) bir `X` için `X.max(axis=0)` şekli (4,), `X.mean(axis=1)` şekli (5,) olur, `X.min()` ise tek bir sayıdır.

Sütun bazında z-skoru: `(X - X.mean(axis=0)) / X.std(axis=0)` ifadesinde (4,) şekilli özetler (5, 4)'ün son boyutuyla hizalanır, bu yüzden ek bir işlem gerekmez. **Satır bazında** standardizasyonda ise (5,) şekilli özet (5, 4) ile hizalanmaz. Bu durumda `keepdims=True` ile (5, 1) şekli korunur. Daha yüksek boyutlu veride hangi eksenin indirgendiğini korumak için `keepdims` güvenli yoldur.

PYTHON
```python
import numpy as np
X = np.array([[1., 2.], [3., 6.], [5., 10.]])
Z = (X - X.mean(axis=0)) / X.std(axis=0)
print(Z.mean(axis=0).round(6), Z.std(axis=0))
print(X.mean(axis=1).shape, X.mean(axis=1, keepdims=True).shape)
```

ÇIKTI
```
[0. 0.] [1. 1.]
(3,) (3, 1)
```

`np.std` varsayılan olarak **ddof=0** kullanır, yani popülasyon standart sapmasını (n'e bölme) hesaplar. Örneklem standart sapması için `ddof=1` verilir (pandas'ın `.std()` metodu varsayılan olarak ddof=1 kullanır):

PYTHON
```python
import numpy as np
print(np.std([1, 2, 3, 4]), np.std([1, 2, 3, 4], ddof=1))
```

ÇIKTI
```
1.118033988749895 1.2909944487358056
```

Vektörleştirilmiş işlemler, iç içe Python döngülerine göre çok daha hızlıdır ve daha az hata riski taşır. Makine öğrenmesi ön işleme hatları milyonlarca değer üzerinde çalıştığı için bu fark belirleyicidir.

Sık yapılan hatalar

- `bin(n)` **çıktısını temiz ikili sanmak:** `0b` öneki vardır. Temiz gösterim için `format(n, 'b')` ya da `bin(n)[2:]` kullanılır.
- **Büyük n'de özyineleme:** derinlik sınırı aşılınca `RecursionError` fırlatılır. Sınırı `sys.setrecursionlimit` ile yükseltmek gerçek bir çözüm değildir.
- `__add__` **içinde tip kontrolü yapmamak:** `other.x` erişimi `AttributeError` verir. Doğrusu `NotImplemented` döndürmektir.
- `NotImplemented` **yerine** `NotImplementedError` **döndürmek ya da fırlatmak:** bu durumda yansıtılmış metot hiç denenmez.
- `sum()` **için** `__radd__` **unutmak:** aşağıdaki gibi `TypeError` alınır.

PYTHON
```python
class V:
    def __init__(self, x):
        self.x = x
    def __add__(self, o):
        return V(self.x + o.x)
```

PYTHON (DEVAM)

```
sum([V(1), V(2)])
```

ÇIKTI

```
TypeError: unsupported operand type(s) for +: 'int' and 'V'
```

- **Sayıları `is` ile karşılaştırmak:** sonuç önbelleğe ve sürüme bağlıdır. Sayılar `==` ile karşılaştırılmalıdır.
- **Önbellek sınırını literal sabitlerle test etmek:** sabit katlama sonucu bozar. Sayılar `int("...")` ile çalışma zamanında üretilmelidir.
- `round(2.5)` **sonucunu 3 sanmak:** sonuç 2'dir (çifte yuvarlama). `round(2.675, 2)` de kayan nokta temsili yüzünden 2.67 verir.
- **(3, 4) + (3,) beklemek:** satır başına bir değer eklemek için sütun vektörü gerekir, yani `reshape(3, 1)` ya da `[:, np.newaxis]`:

PYTHON

```
import numpy as np
np.ones((3, 4)) + np.array([1, 2, 3])
```

ÇIKTI

```
ValueError: operands could not be broadcast together with shapes (3,4) (3,)
```

- **axis yönünü karıştırmak:** sütun ortalaması `axis=0` ile alınır.

Kısa tekrar soruları

1. `format(6, 'b')` ve `bin(6)` ne döndürür?
2. 0..n arasını yazan özyinelemeli çözümün yığın belleği nedir, döngülü çözümünki nedir?
3. `__add__` `NotImplemented` döndürürse Python ne yapar?
4. `sum([v1, v2])` neden `__radd__` gerektirir?
5. `a = int("257"); b = int("257")` için `a == b` ve `a is b` ne verir (bu ortamda)?
6. Task 3'te tamsayılar neden çalışma zamanında üretilmelidir?
7. `round(0.5)`, `round(1.5)`, `int(-2.7)` ne verir?
8. Banker's rounding hangi sorunu azaltır?
9. (3, 4) + (3, 1) ve (3, 4) + (3,) işlemlerinin sonucu nedir?
10. `np.std` varsayılan olarak hangi ddof değerini kullanır, `keepdims=True` ne zaman gerekir?

Cevaplar

1 `'110'` ve `'0b110'`.

2 Özyinelemeli çözüm O(n) kullanır (her çağrı bir frame), döngülü çözüm O(1) ek bellek kullanır.

3 Diğer operandın `__radd__` metodunu dener. O da `NotImplemented` dönerse ya da yoksa `TypeError` fırlatılır.

4 `sum` 0'dan başlar ve ilk işlem `0 + v1` olur. `int` Vector'ü bilmediği için `v1.__radd__(0)` çağrılır.

5 `True False` (bu ortamda, CPython 3.9.6, çalıştırıldı). 257 önbellek aralığı -5…256'nın dışındadır.

6 Aynı kod bloğundaki literal sabitler derleyicide tek nesneye birleşebilir (sabit katlama). Bu durumda `is` önbellek dışındaki sayılarda bile True çıkar ve sınır yanlış ölçülür.

7 `0`, `2`, `-2`.

8 Tam .5'li değerlerde hep yukarı yuvarlamanın yarattığı sistematik sapmayı (bias) azaltır.

9 İlki (3, 4) verir. İkincisi `ValueError` verir, çünkü sondan karşılaştırmada 4 ile 3 uyuşmaz.

10 ddof=0 kullanır. `keepdims=True`, satır bazında (ya da daha yüksek boyutta ara bir eksende) indirgenen özetin orijinal diziyle yeniden broadcast edilmesi gerektiğinde, yani eksenin yerinde 1 kalması gerektiğinde kullanılır.